

BCAC 0026: ANALYSIS OF ALGORITHM AND DATA STRUCTURES

Faculty Name

Dr. Vinod Jain, Assistant Professor, CEA

Mob : 9813078438, Email – vinod.jain@gla.ac.in

Seating : 329-B, AB-X, Third Floor

Sorting Techniques

- **Sorting Techniques:** Elementary sorting techniques-Bubble Sort, Insertion Sort, Merge Sort, Advanced Sorting techniques-Heap Sort, Quick Sort, Sorting in Linear Time-Bucket Sort, Radix Sort and Count Sort.

Sorting Techniques

- **Elementary sorting techniques-**

1. Bubble Sort
2. Insertion Sort
3. Selection Sort

- **Advanced Sorting techniques-**

1. Merge Sort
2. Quick Sort,

Sorting Techniques

- A **Sorting Algorithm** is used to rearrange a given array or list of elements in an order.
- For Example initially the array is
- 5,6,1,3
- After sorting it will become
- 1,3,5,6
- OR
- 6,5,3,1

Sorting Techniques

- **Elementary sorting techniques are**

1. Bubble Sort
2. Insertion Sort
3. Merge Sort
4. Selection Sort

Bubble Sort

A collection of colorful, iridescent bubbles of various sizes against a black background. The bubbles are arranged in a way that suggests a sorting process, with some bubbles appearing to be in motion or about to burst. The colors range from deep blues and purples to bright yellows and oranges, creating a vibrant contrast with the dark background.

Bubble Sort

Bubble Sort

- **Bubble Sort** is the simplest [sorting algorithm](#) that works by repeatedly swapping the adjacent elements if they are in the wrong order.
- This algorithm is not suitable for large data sets as its average and worst-case time complexity are quite high.

Bubble Sort

- It iterates across the array and compares nearby elements.
- A swap is carried out if the current element is larger than the following element.
- Until the largest element "bubbles" to the end of the array, this process is repeated.

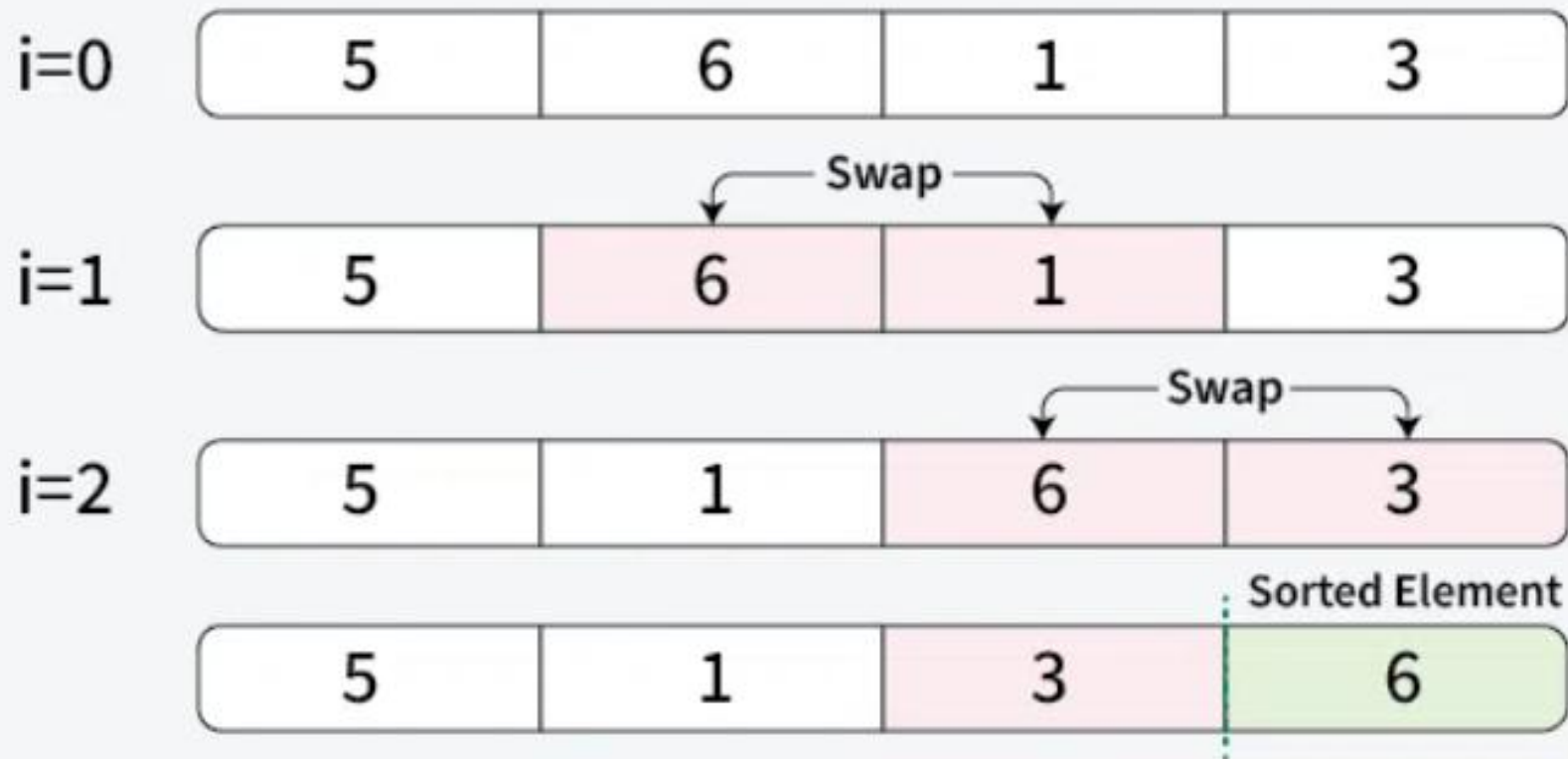
Bubble Sort

- We sort the array using multiple passes. After the first pass, the maximum element goes to end (its correct position).
- Same way, after second pass, the second largest element goes to second last position and so on.

01

Step

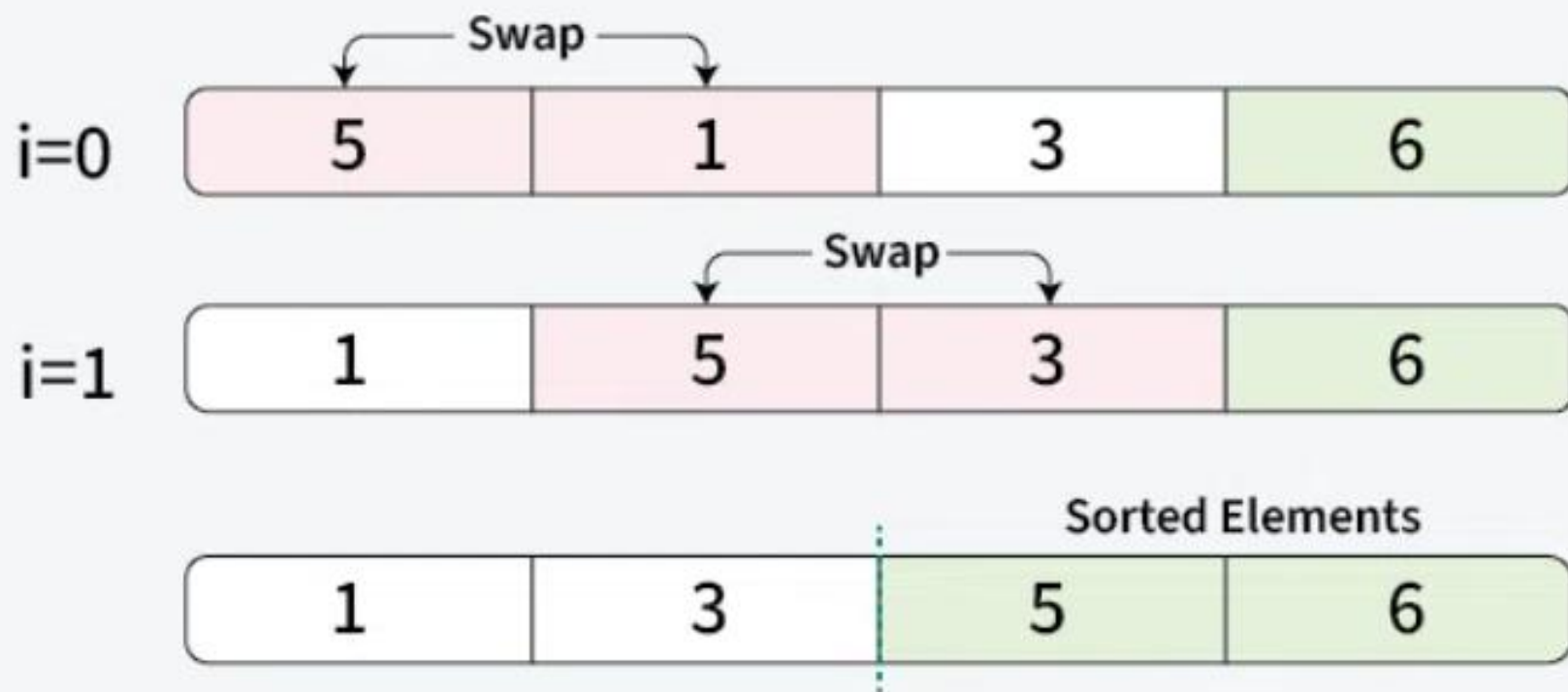
Placing the 1st largest element at its correct position



Bubble sort

02
Step

Placing 2nd largest element at its correct position



Bubble sort

03
Step

Placing 3rd largest element at its correct position



Bubble sort

Algorithm 1: Bubble sort

Data: Input array $A[]$

Result: Sorted $A[]$

$int\ i, j, k;$

$N = length(A);$

for $j = 1$ **to** N **do**

for $i = 0$ **to** $N-1$ **do**

if $A[i] > A[i+1]$ **then**

$temp = A[i];$

$A[i] = A[i+1];$

$A[i+1] = temp;$

end

end

end

```
void Sort_Array_Using_Bubble_Sort_2()  
{  
    n = x.length;  
    int i, j;  
    for (i=0; i<n-1; i++)  
    {  
        for (j=0; j<n-i-1; j++)  
        {  
            if (x[j] > x[j+1])  
            {  
                int temp = x[j];  
                x[j] = x[j+1];  
                x[j+1] = temp;  
            }  
        }  
    }  
}
```

Bubble Sort

- Optimization in Bubble sort
- If no element is swapped in any pass, it means array is already sorted and stop further passes.
- Take a Boolean variable swapped for it.
- For example

Standard Algorithm for Bubble Sort

Algorithm: BubbleSort(arr, n)

Sorts an array `arr` of size `n` in ascending order.

Input:

- `arr`: Array of integers (unsorted).
- `n`: Size of the array.

Output:

- `arr`: Array of integers (sorted in ascending order).

Steps:

1. **Start**
2. Input the array `arr` and its size `n`.
3. **For** $i = 0$ to $n - 2$:
 - Set a flag `swapped` to `false`.
 - **For** $j = 0$ to $n - i - 2$:
 - **If** $arr[j] > arr[j + 1]$:
 - Swap $arr[j]$ and $arr[j + 1]$.
 - Set `swapped` to `true`.
 - **If** `swapped` is `false`, break the loop (array is sorted).
4. **End For**
5. The array `arr` is now sorted.
6. Output the sorted array.
7. **End**

```
public static void bubbleSort(int[] array) {  
    int n = array.length;  
    boolean swapped; // To optimize: check if swapping happened  
    for (int i = 0; i < n - 1; i++) {  
        swapped = false;  
        // Loop through the array, excluding the sorted elements  
        for (int j = 0; j < n - i - 1; j++) {  
            if (array[j] > array[j + 1]) {  
                // Swap adjacent elements if out of order  
                int temp = array[j];  
                array[j] = array[j + 1];  
                array[j + 1] = temp;  
                swapped = true;  
            }  
        }  
        // If no elements were swapped, array is already sorted  
        if (!swapped) {  
            break;  
        }  
    }  
}
```

Bubble Sort Program in Java : Font 6

```

class BubbleSort
{
    int x[]={4,2,7,9,1,5,8,11,3};
    int n;
    void print_array()
    {
        n = x.length;
        System.out.println("\n ");
        for(int i=0;i<n;i++)
        {
            System.out.print(" " + x[i]);
        }
    }
    void sort_array_using_bubble_sort()
    {
        n = x.length;
        int i,j,temp;
        for(i=0;i<n-1;i++)
        {
            for(j=0;j<n-1-i;j++)
        
```

```

        {
            if(x[j] > x[j+1])
            {
                //swap
                temp=x[j];
                x[j]=x[j+1];
                x[j+1] =temp;
            }
        }
        void sort_array_using_bubble_sort2()
        {
            n = x.length;
            int i,j,temp;
            for(i=0;i<n-1;i++)
            {
                this.print_array();
                for(j=0;j<n-1-i;j++)
            
```

```

        {
            if(x[j] > x[j+1])
            {
                //swap
                System.out.format("\n Swapped %d*
<-> %d ", x[j],x[j+1]);
                temp=x[j];
                x[j]=x[j+1];
                x[j+1] =temp;
            }
        }
    }
}

BubbleSort bs = new BubbleSort();
System.out.println("\n Before Sorting \n");
bs.print_array();

bs.sort_array_using_bubble_sort2();

System.out.println("\n After Sorting \n");
bs.print_array();

System.out.println("\n\n");
}
}

public class BubbleSortingDemo
{
    public static void main(String args[])
    {
    
```

```
// Bubble Sort
import java.util.Arrays;
public class BubbleSorting
{
    public static void main(String args[])
    {
        int x[] = {6,2,5,1,3};
        int i,j,n,temp;
        n = x.length;
        System.out.print("\n Unsorted array " + Arrays.toString(x));
        for(i=0;i<n-1;i++)
        {
            for(j=0;j<n-1-i;j++)
            {
                if(x[j] > x[j+1])
                {
                    temp=x[j];
                    x[j]=x[j+1];
                    x[j+1] = temp;
                }
            }
        }
        System.out.print("\n Sorted array " + Arrays.toString(x));
    }
}
```

Bubble Sort

- Complexity Analysis of Bubble Sort:
- **Time Complexity:** $O(n^2)$
Auxiliary Space: $O(1)$

Bubble Sort

- **Advantages of Bubble Sort:**
- Bubble sort is easy to understand and implement.
- It does not require any additional memory space.
- It is a stable sorting algorithm, meaning that elements with the same key value maintain their relative order in the sorted output.
- It is the best algorithm if array is already sorted. Stop sorting after first pass by using a flag swap=true or false.

Bubble Sort

- **Disadvantages of Bubble Sort:**
- Bubble sort has a time complexity of $O(n^2)$ which makes it very slow for large data sets.

Insertion Sort

The Idea of the insertion sort is similar to the Idea of sorting the Playing cards .

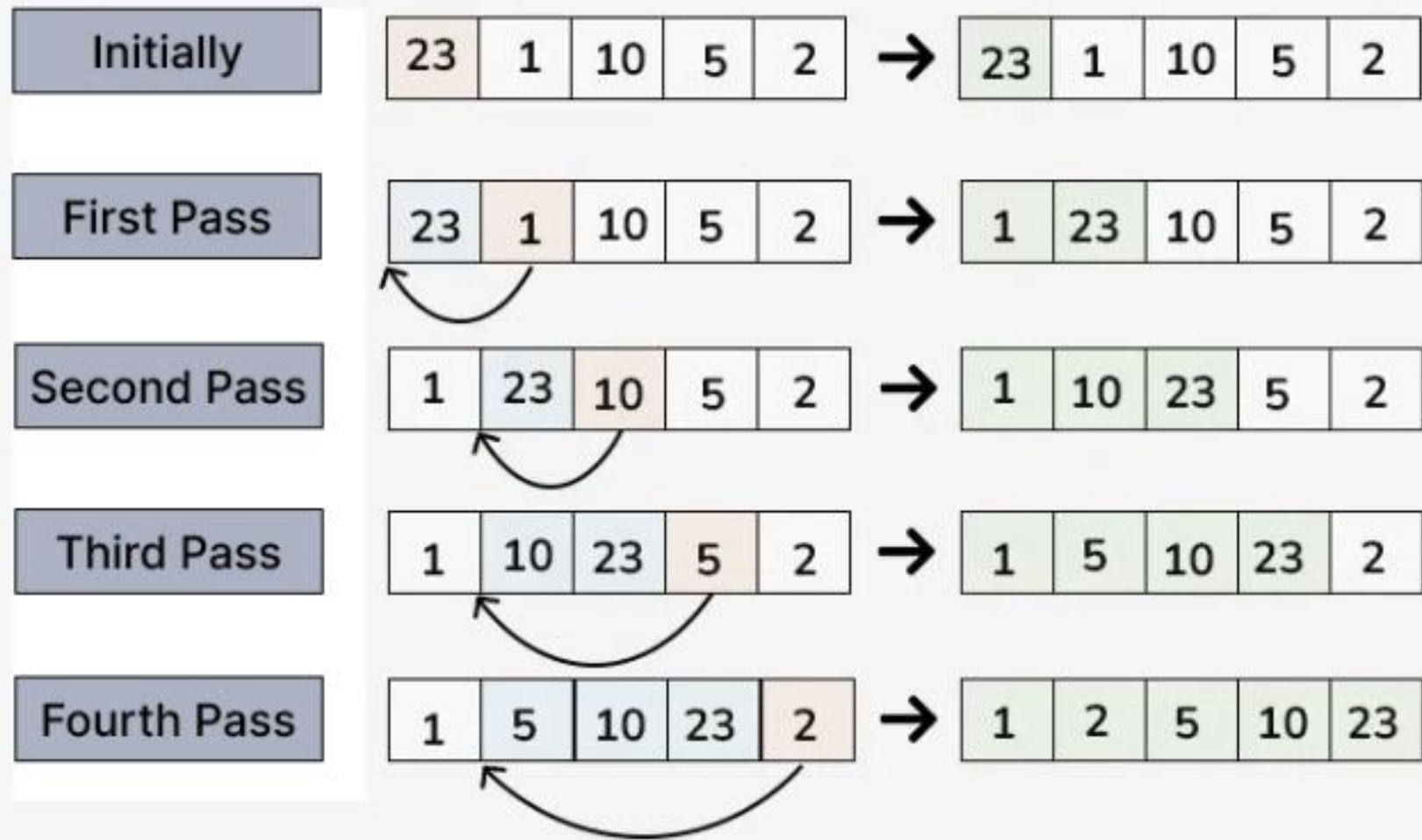


Insertion Sort

- <https://www.geeksforgeeks.org/insertion-sort-algorithm/>

Insertion Sort

- **Insertion sort** is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list.
- It is like sorting playing cards in your hands.
- You split the cards into two groups: the sorted cards and the unsorted cards.
- Then, you pick a card from the unsorted group and put it in the right place in the sorted group.



Insertion Sort



Insertion Sort

- We start with second element of the array as first element in the array is assumed to be sorted.
- Compare second element with the first element and check if the second element is smaller then swap them.
- Move to the third element and compare it with the first two elements and put at its correct position
- Repeat until the entire array is sorted.

Insertion Sort

- Example:
- Let's use the same array of integers: {5, 2, 8, 12, 3}.
- In the first iteration, the algorithm considers 5 as the first element, which is already sorted, so no changes occur.
- In the second iteration, the algorithm considers 2 and inserts it before 5, resulting in {2, 5, 8, 12, 3}.
- In the third iteration, the algorithm takes into account 8, which is already in the right place.
- In the fourth iteration, the algorithm takes into account 12, which is already in the right position.
- In the fifth iteration, the algorithm considers 3 and inserts it between 2 and 5, resulting in {2, 3, 5, 8, 12}.

Insertion Sort

- https://www.tutorialspoint.com/data_structures_algorithms/insertion_sort_algorithm.htm

Insertion Sort Algorithm

Now we have a bigger picture of how this sorting technique works, so we can derive simple steps by which we can achieve insertion sort.

Step 1 – If it is the first element, it is already sorted. return 1;

Step 2 – Pick next element

Step 3 – Compare with all elements in the sorted sub-list

Step 4 – Shift all the elements in the sorted sub-list that is greater than the value to be sorted

Step 5 – Insert the value

Step 6 – Repeat until list is sorted

Pseudocode

```
Algorithm: Insertion-Sort(A)
for j = 2 to A.length
    key = A[j]
    i = j - 1
    while i > 0 and A[i] > key
        A[i + 1] = A[i]
        i = i - 1
    A[i + 1] = key
```

```
void Sort_Array_Using_insertion_Sort()
{
    n = x.length;
    int i, j;
    for(i=1; i<n; i++) // Set i=1
    {
        int key = x[i];
        j=i-1;
        while(j>=0 && x[j] > key)
        {
            x[j+1] = x[j];
            j=j-1;
        }
        x[j+1] = key;
    }
}
```

```
// Insertion Sort
import java.util.Arrays;
public class InsertionSortv5
{
    public static void main(String args[])
    {

        int x[] = {6,2,5,1,3};
        int i,j,n,key;
        n = x.length;
        System.out.print("\n Unsorted array " + Arrays.toString(x));
        for(i=1;i<n;i++)
        {
            key = x[i];
            for(j=i-1;j>=0;j--)
            {
                if(x[j] > key) // shit it to right
                {
                    x[j+1] = x[j];
                }
                else
                {
                    break; // stop shifting
                }
            }
            x[j+1] = key;
        }
        System.out.print("\n Sorted array " + Arrays.toString(x));
    }
}
```

Insertion Sort

- <https://www.geeksforgeeks.org/insertion-sort-algorithm/>

Complexity Analysis of Insertion Sort :

- **Time Complexity of Insertion Sort**
- **Best case: $O(n)$** , If the list is already sorted, where n is the number of elements in the list.
- **Average case: $O(n^2)$** , If the list is randomly ordered
- **Worst case: $O(n^2)$** , If the list is in reverse order
- **Space Complexity of Insertion Sort**
- **Auxiliary Space: $O(1)$** , Insertion sort requires **$O(1)$** additional space, making it a space-efficient sorting algorithm.

Advantages of Insertion Sort:

- Simple and easy to implement.
- **Stable** sorting algorithm.
- Efficient for small lists and nearly sorted lists.
- Space-efficient as it is an in-place algorithm.
- Adoptive. the [number of inversions](#) is directly proportional to number of swaps. For example, no swapping happens for a sorted array and it takes $O(n)$ time only.

Disadvantages of Insertion Sort:

- Inefficient for large lists.
- Not as efficient as other sorting algorithms (e.g., merge sort, quick sort) for most cases.

Insertion Sort: Program in Java

```

• /*
• * To change this license header,
• choose License Headers in Project
• Properties.
•
• * To change this template file, choose
• Tools | Templates
•
• * and open the template in the editor.
• */
• package p1;
• import java.util.Scanner;
• class SortingClass
• {
•     int x[] = {5,1,3,7,4,8,2};
•     int n;
•     void print_array()
•     {
•         System.out.print("\n Array is : ");
•         for(int item : x)
•         {
•             System.out.print(" " + item);
•         }
•     }
•     void read_array()
•     {
•         Scanner sc = new
•         Scanner(System.in);
•         System.out.print("\n Enter n : ");
•         n = sc.nextInt();
•         x = new int[n];
•         System.out.print("\n Enter Array
•         Elements : ");
•         for(int i=0;i<n;i++)
•         {
•             x[i] = sc.nextInt();
•         }
•         this.print_array();
•     }
•     void
•     Sort_Array_Using_insertion_Sort()
•     {
•         n = x.length;
•         for(i=1;i<n;i++)
•         {
•             int i,j;
•             for(i=1;i<n;i++) // Set i=1
•             {
•                 int key = x[i];
•                 j=i-1;
•                 while(j>=0&& x[j] > key)
•                 {
•                     x[j+1] = x[j];
•                     j=j-1;
•                 }
•                 x[j+1] = key;
•             }
•             void
•             Sort_Array_Using_insertion_Sort2()
•             {
•                 n = x.length;
•                 int i,j;
•                 for(i=1;i<n;i++)
•                 {
•                     this.print_array();
•                     int item = x[i];
•                     insert " + item);
•                     ReadData.readkey();
•                     for(j=i-1;j>=0 && x[j] > item;j--)
•                     {
•                         x[j+1] = x[j];
•                     }
•                     x[j+1] = item;
•                     this.print_array();
•                     ReadData.readkey();
•                     // Swap item
•                 }
•             }
•         }
•     }
•     public static void main(String args[])
•     {
•         SortingClass sc = new
•         SortingClass();
•         //sc.read_array();
•         System.out.print("\n\n Initial
•         Array ");
•         sc.print_array();
•         //sc.Sort_Array_Using_Bubble_Sort();
•         sc.Sort_Array_Using_insertion_Sort2();
•         //sc.Sort_Array_Using_Bubble_Sort_2(
•         );
•         System.out.print("\n\n Final
•         Sorted Array ");
•         sc.print_array();
•         System.out.print("\n");
•     }
• }

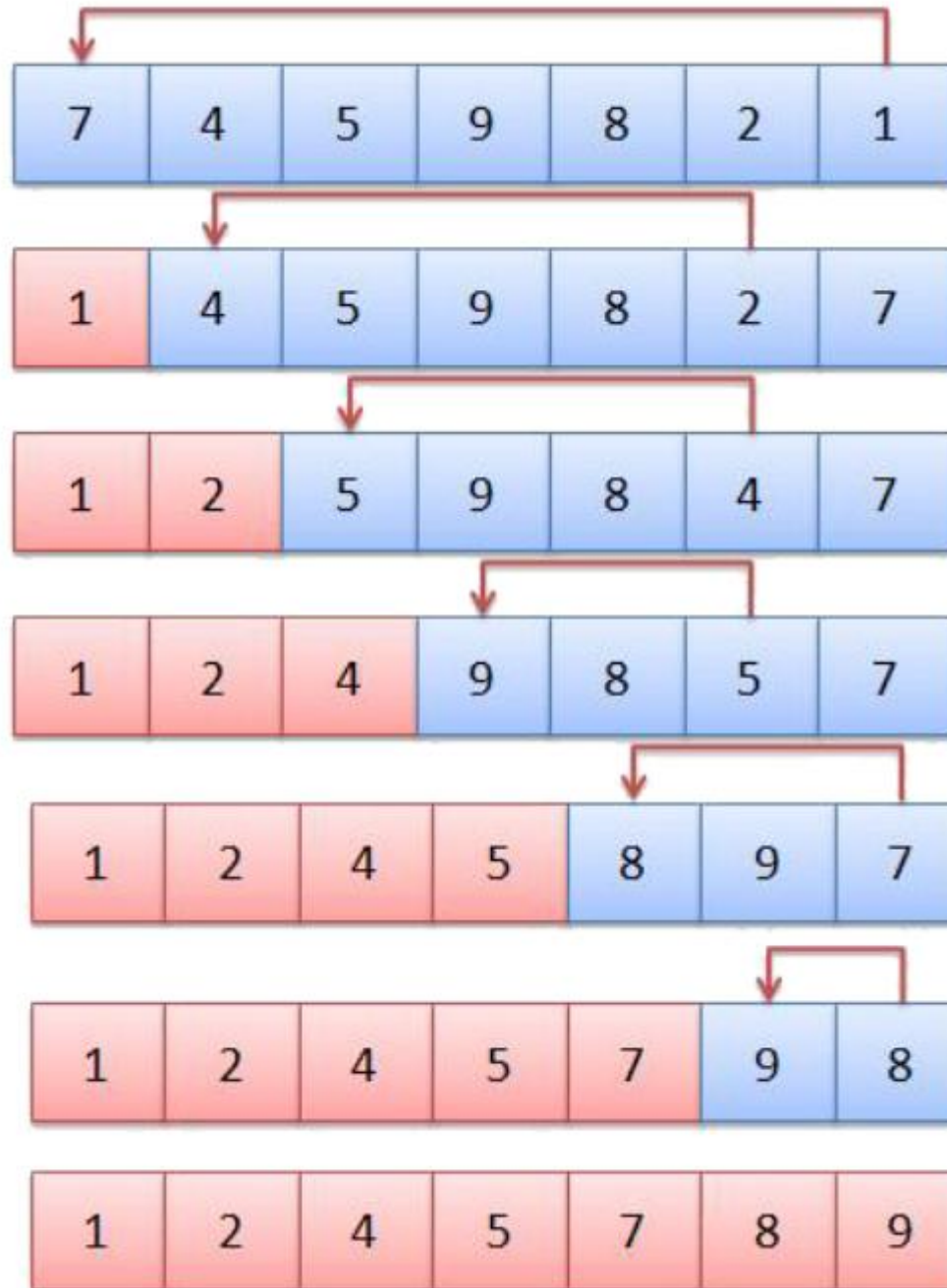
```

Selection Sort

Selection Sort

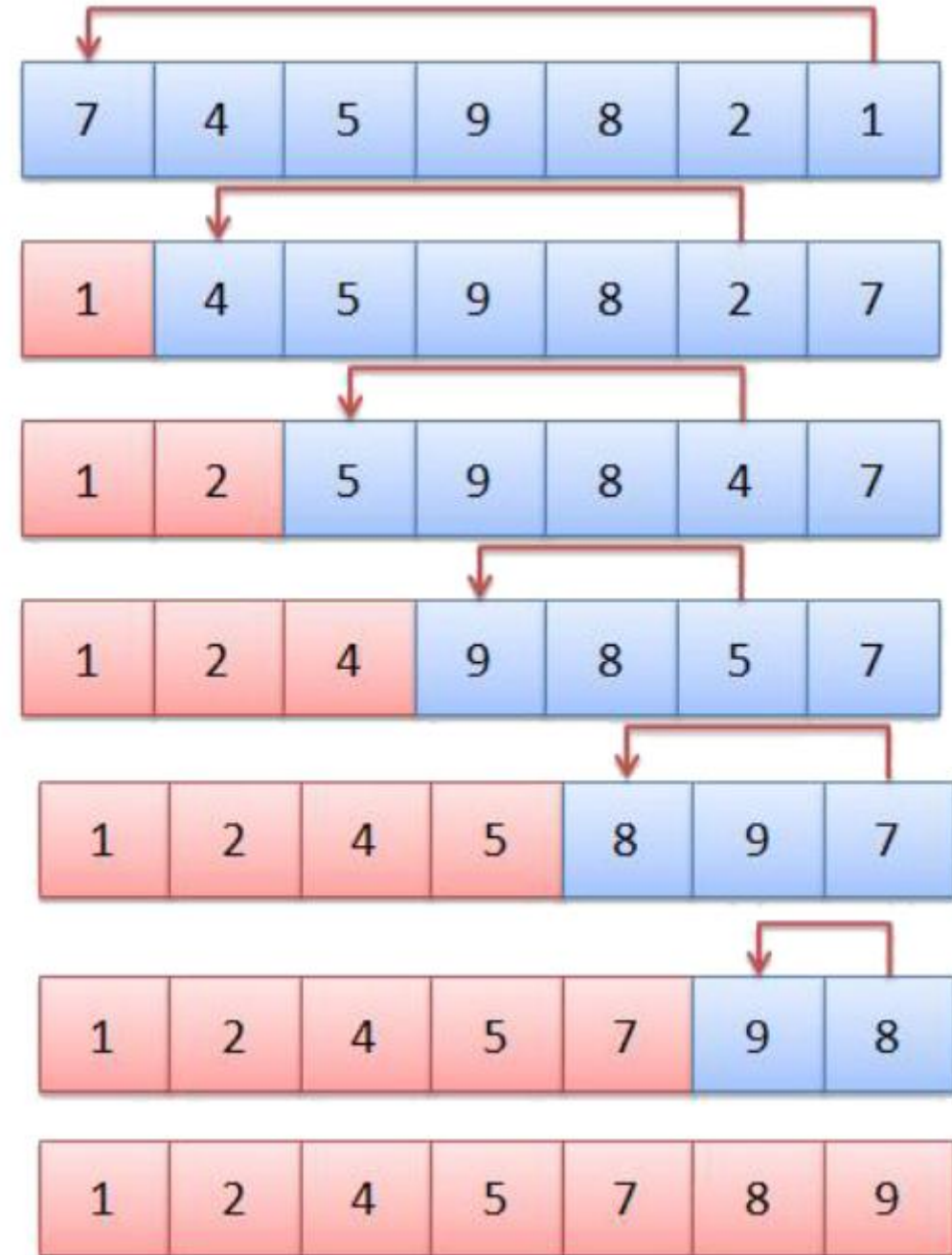
- **Selection Sort** is a comparison-based sorting algorithm.
- It sorts an array by repeatedly selecting the **smallest** element from the unsorted portion and swapping it with the first unsorted element.
- This process continues until the entire array is sorted.

Selection Sort



Selection Sort

1. First we find the smallest element and swap it with the first element. This way we get the smallest element at its correct position.
2. Then we find the smallest among remaining elements (or second smallest) and swap it with the second element.
3. We keep doing this until we get all elements moved to correct position.

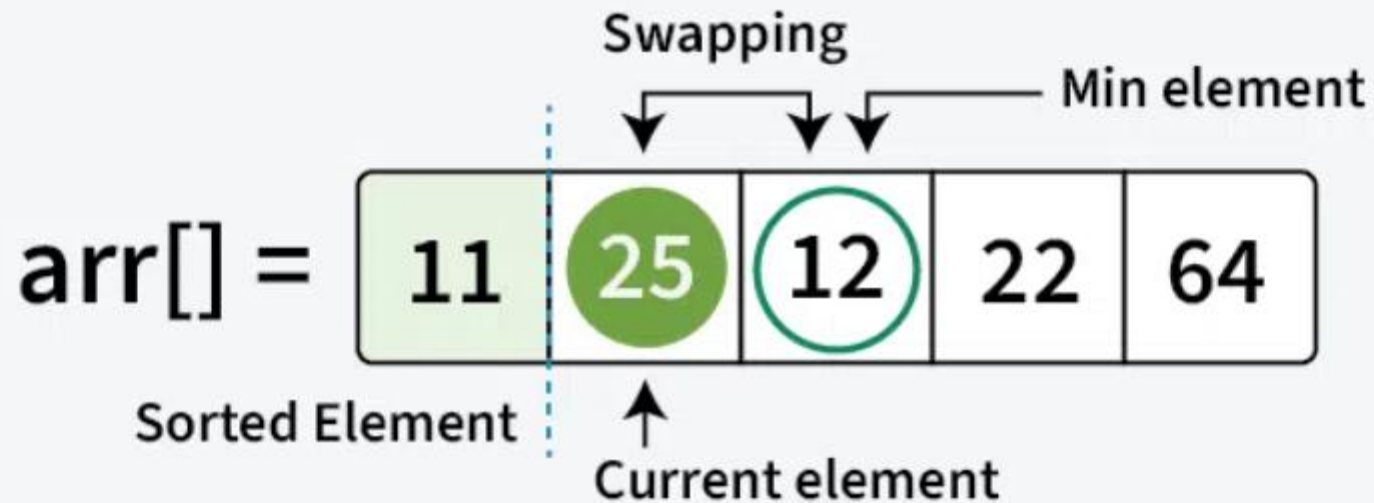


Start from the first element at index 0, find the smallest element in the rest of the array which is unsorted, and swap (11) with current element(64).



02
Step

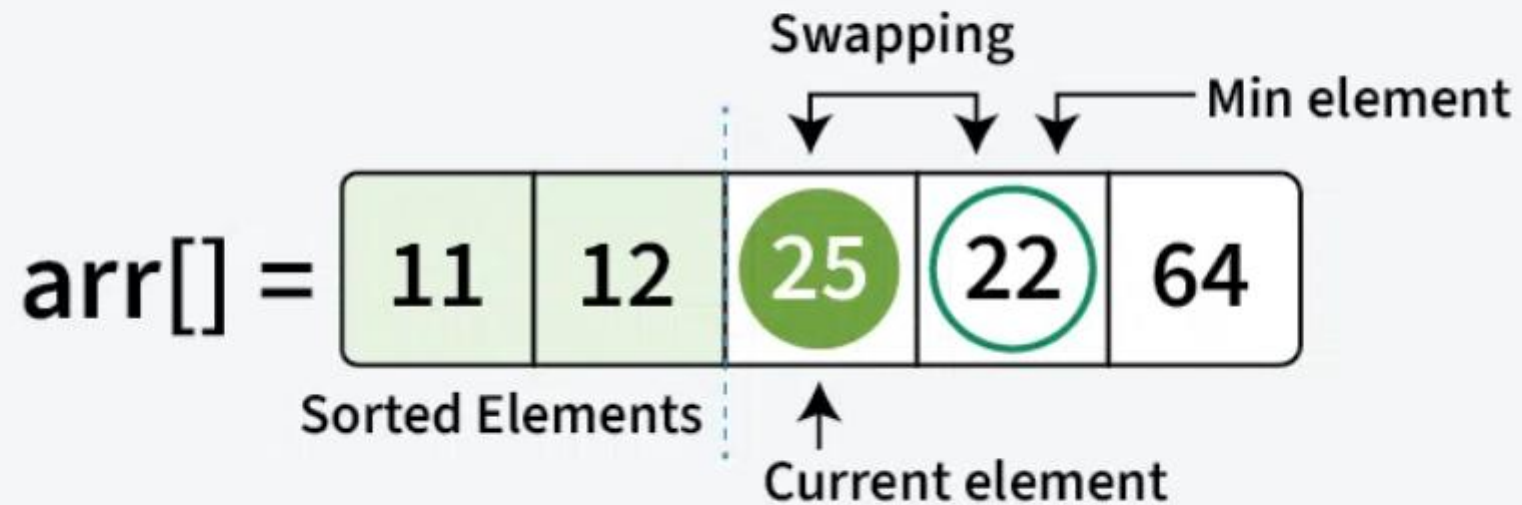
Move to the next element at index 1 (25). Find the smallest in unsorted subarray, and swap (12) with current element (25).



Selection Sort Algorithm

03
Step

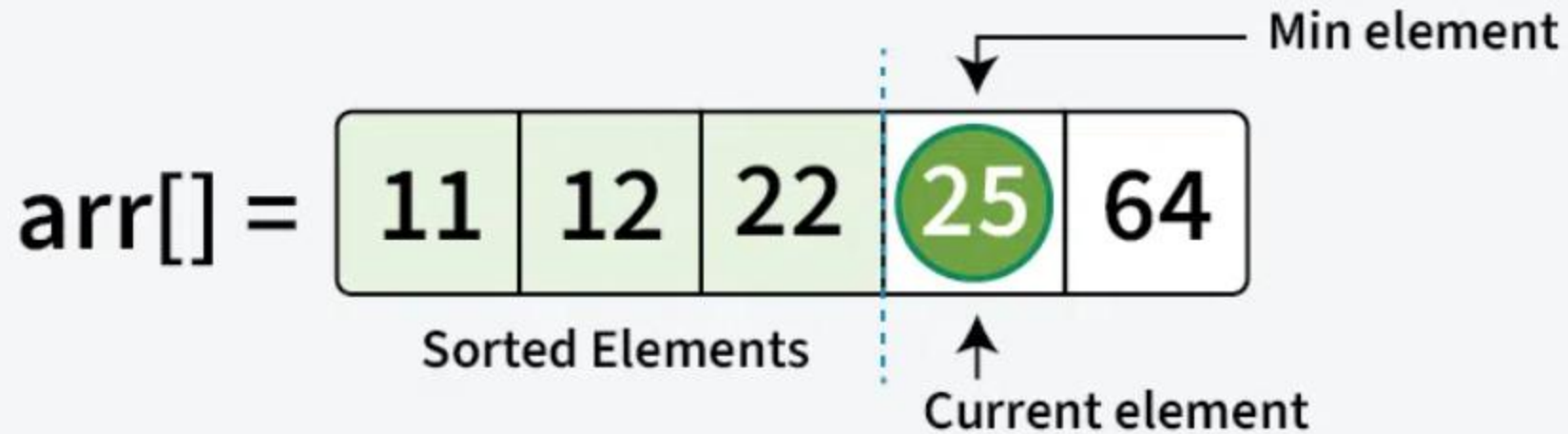
Move to element at index 2 (25). Find the minimum element from unsorted subarray, Swap (22) with current element (25).



Selection Sort Algorithm

04
Step

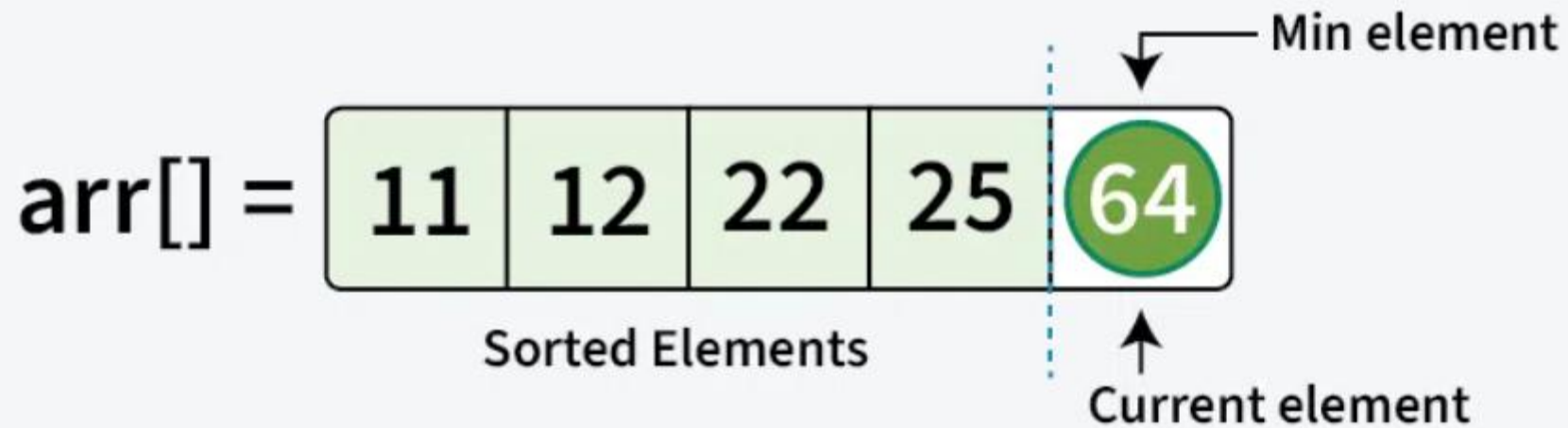
Move to element at index 3 (25), find the minimum from unsorted subarray and swap (25) with current element (25).



Selection Sort Algorithm

05
Step

Move to element at index 4 (64), find the minimum from unsorted subarray and swap (64) with current element (64).



Selection Sort Algorithm

06

Step

We get the sorted array at the end.

arr[] =

11	12	22	25	64
----	----	----	----	----

Sorted array

Algorithm

SELECTION SORT(arr, n)

Step 1: Repeat Steps 2 **and** 3 **for** $i = 0$ to $n-1$

Step 2: CALL SMALLEST(arr, i, n, pos)

Step 3: SWAP arr[i] with arr[pos]

[END OF LOOP]

Step 4: EXIT

SMALLEST (arr, i, n, pos)

Step 1: [INITIALIZE] SET SMALL = arr[i]

Step 2: [INITIALIZE] SET pos = i

Step 3: Repeat **for** $j = i+1$ to n

if (SMALL > arr[j])

 SET SMALL = arr[j]

 SET pos = j

[END OF **if**]

[END OF LOOP]

Step 4: RETURN pos

Selection Sort

SELECTION SORT(arr, n)

Step 1: Repeat Steps 2 **and** 3 **for** $i = 0$ to $n-1$

Step 2: CALL SMALLEST(arr, i, n, pos)

Step 3: SWAP arr[i] with arr[pos]

[END OF LOOP]

Step 4: EXIT

SMALLEST (arr, i, n, pos)

Step 1: [INITIALIZE] SET SMALL = arr[i]

Step 2: [INITIALIZE] SET pos = i

Step 3: Repeat **for** $j = i+1$ to n

if (SMALL > arr[j])

 SET SMALL = arr[j]

 SET pos = j

[END OF **if**]

[END OF LOOP]

Step 4: RETURN pos

This is the pseudocode I used:

Algorithm 4 Selection Sort

```
1: for  $i = 1$  to  $n - 1$  do  
2:    $min = i$   
3:   for  $j = i + 1$  to  $n$  do  
4:     // Find the index of the  $i^{th}$  smallest element  
5:     if  $A[j] < A[min]$  then  
6:        $min = j$   
7:     end if  
8:   end for  
9:   Swap  $A[min]$  and  $A[i]$   
10: end for
```

```
for (i=0; i<n-1; i++)  
{  
    int pos=i;  
    int min = x[i];  
    for (j=i+1; j<n; j++)  
    {  
        if (x[j] < min)  
        {  
            min = x[j];  
            pos=j;  
        }  
    }  
    int temp = x[i];  
    x[i] = x[pos];  
    x[pos] = temp;  
}
```

```
// Sort an array using Selection sort
import java.util.Arrays;

public class SelectionSortV1
{
    public static void main(String aargs[])
    {
        int key,i,j;
        int x[] = { 23,1,10,5,2};
        System.out.print("\n UnSorted array is : " + Arrays.toString(x));
        int n = x.length;
        for(i=0;i<n-1;i++)
        {
            int pos=i;
            int min = x[i];
            for(j=i+1;j<n;j++)
            {
                if(x[j] < min)
                {
                    min = x[j];
                    pos=j;
                }
            }
            int temp = x[i];
            x[i] = x[pos];
            x[pos] = temp;
        }
        System.out.print("\n Sorted array is : " + Arrays.toString(x));
        System.out.print("\n\n\n\n ");
    }
}
```


Selection Sort

- **Complexity Analysis of Selection Sort**
- **Time Complexity: $O(n^2)$** ,as there are two nested loops:
- One loop to select an element of Array one by one = $O(n)$
- Another loop to compare that element with every other Array element = $O(n)$
- Therefore overall complexity = $O(n) * O(n) = O(n*n) = O(n^2)$
- **Auxiliary Space: $O(1)$** as the only extra memory used is for temporary variables.

Selection Sort

- **Advantages of Selection Sort**
- Easy to understand and implement, making it ideal for teaching basic sorting concepts.
- Requires only a constant $O(1)$ extra memory space.
- It requires less number of swaps (or memory writes) compared to many other standard algorithms. Only [cycle sort](#) beats it in terms of memory writes. Therefore it can be simple algorithm choice when memory writes are costly.

Selection Sort

- **Disadvantages of the Selection Sort**
- Selection sort has a time complexity of $O(n^2)$ makes it slower compared to algorithms like [Quick Sort](#) or [Merge Sort](#).
- Does not maintain the relative order of equal elements which means it is not stable.

Selection Sort

- **Applications of Selection Sort**
- Perfect for teaching fundamental sorting mechanisms and algorithm design.
- Suitable for small lists where the overhead of more complex algorithms isn't justified and memory writing is costly as it requires less memory writes compared to other standard sorting algorithms.

Selection Sort

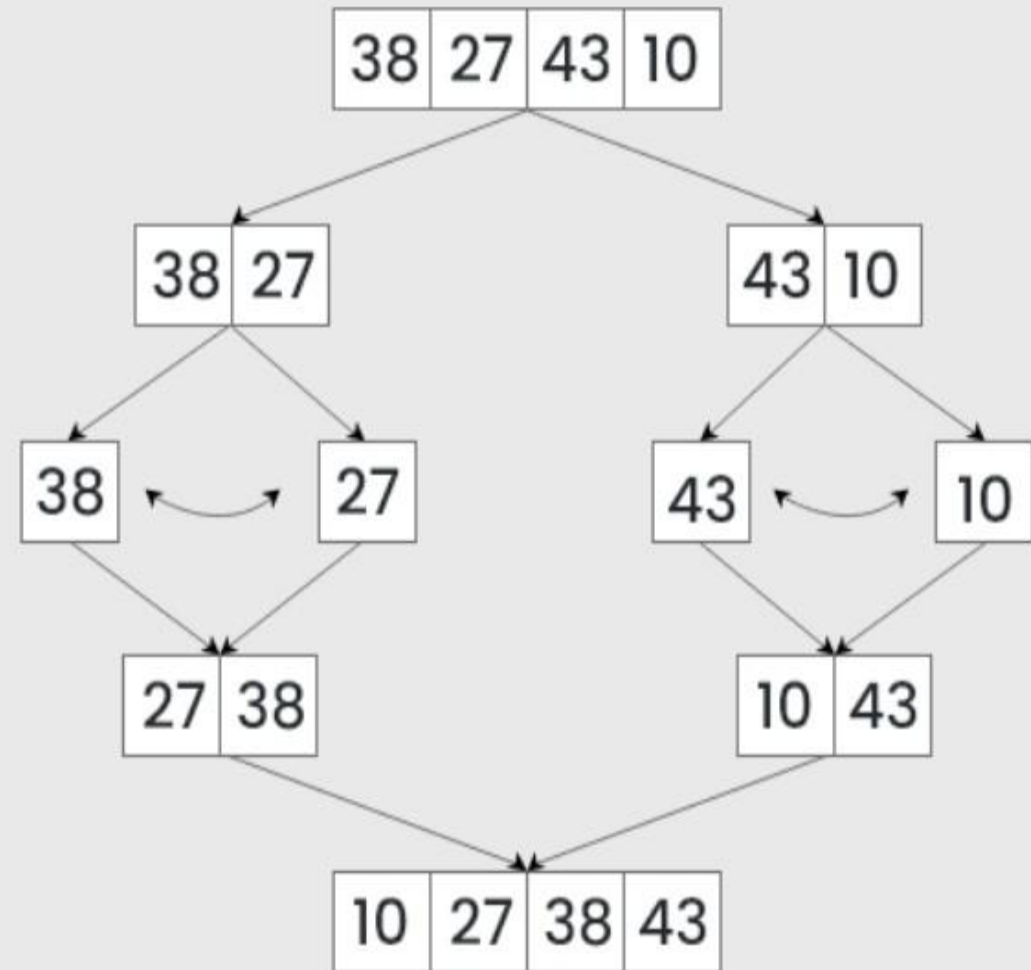
- <https://cs.stackexchange.com/questions/139317/why-is-my-runtime-calculation-of-selection-sort-different>
- <https://www.geeksforgeeks.org/selection-sort-algorithm-2/>
- <https://www.javatpoint.com/selection-sort>

- Advanced Sorting techniques –
- Merge Sort, Heap Sort, Quick Sort
- Sorting in Linear Time
- -Bucket Sort, Radix Sort and Count Sort.

Merge Sort

Merge Sort

Algorithm



Divide and Conquer Algorithm

- **Divide and Conquer algorithm** is a problem-solving strategy that involves.
- **Divide** : Break the given problem into smaller non-overlapping problems.
- **Conquer** : Solve Smaller Problems
- **Combine** : Use the Solutions of Smaller Problems to find the overall result.
- Examples of Divide and Conquer are Merge Sort, Quick Sort, Binary Search and many more.

Merge Sort

- **How does Merge Sort work?**
- Merge sort is a popular sorting algorithm known for its efficiency and stability.
- It follows the **divide-and-conquer** approach to sort a given array of elements.

Merge Sort

- Here's a step-by-step explanation of how merge sort works:
- Divide: Divide the list or array recursively into two halves until it can no more be divided.
- Conquer: Each subarray is sorted individually using the merge sort algorithm.
- Merge: The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Merge Sort

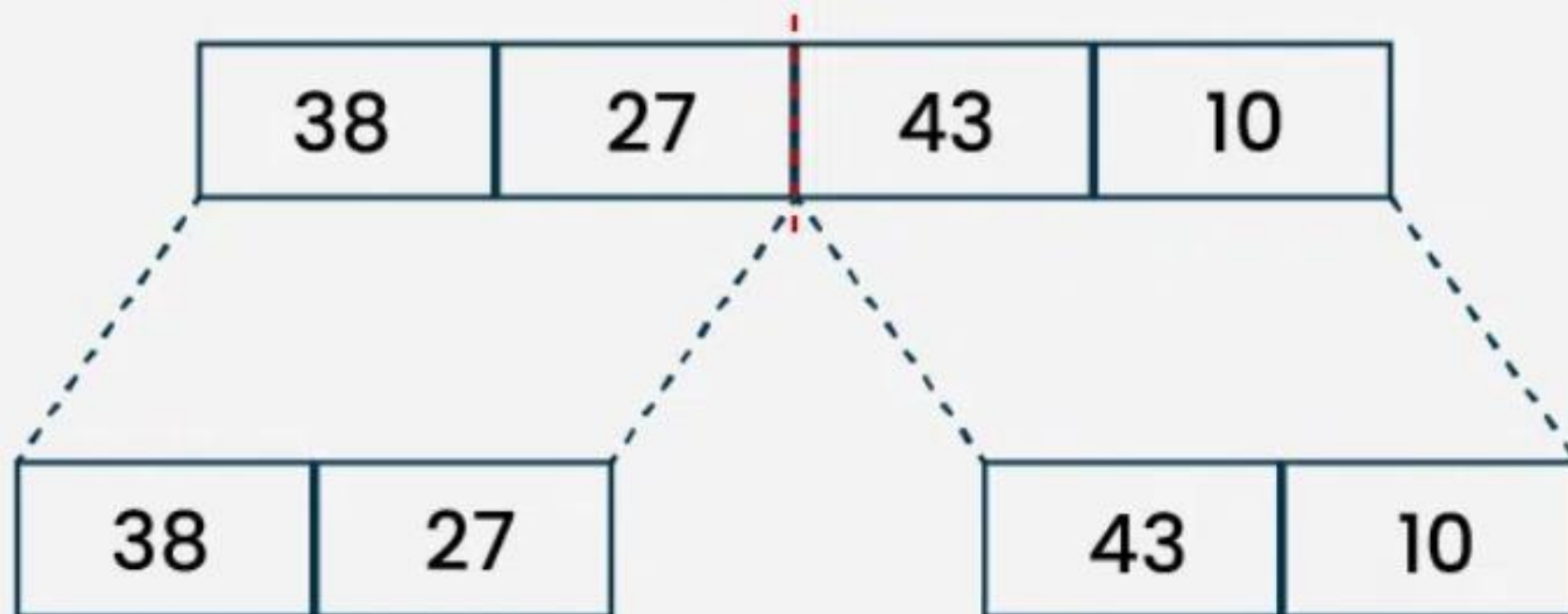
- **Illustration of Merge Sort:**
- Let's sort the array or list **[38, 27, 43, 10]** using Merge So



Step 1

Splitting the Array into two equal halves

Partition

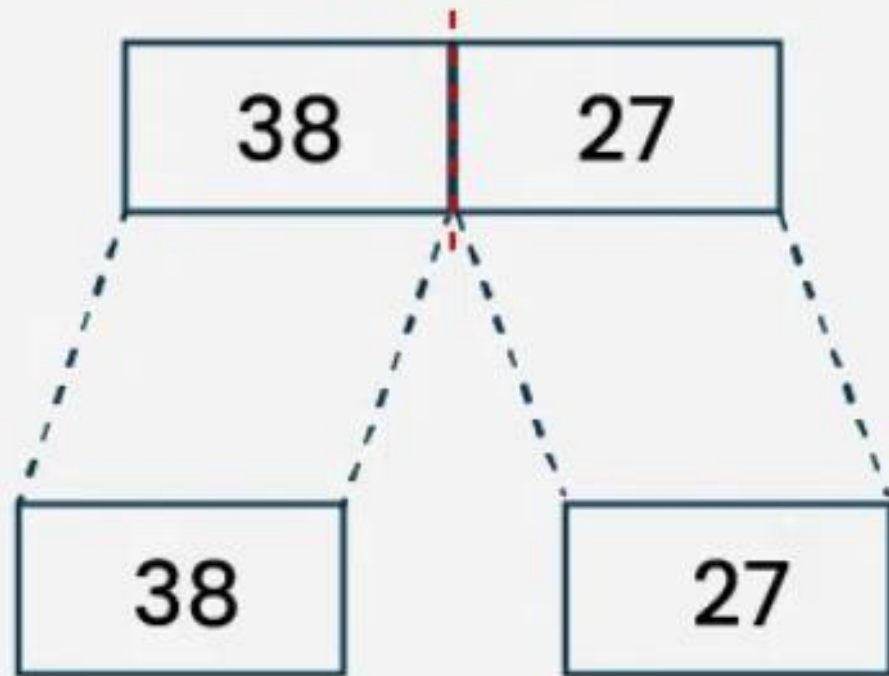




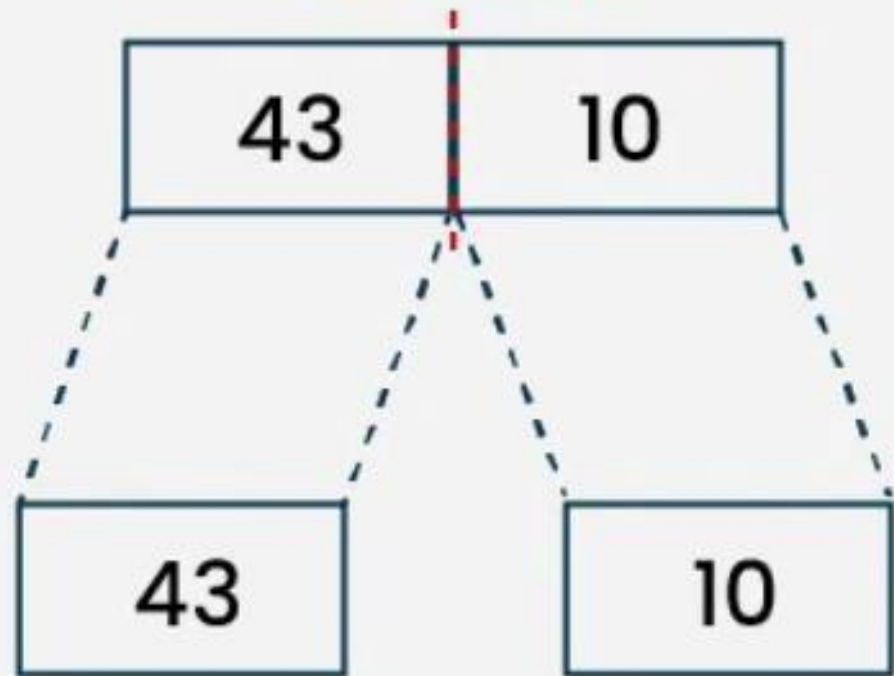
Step 2

Splitting the subarrays into two halves

Partition



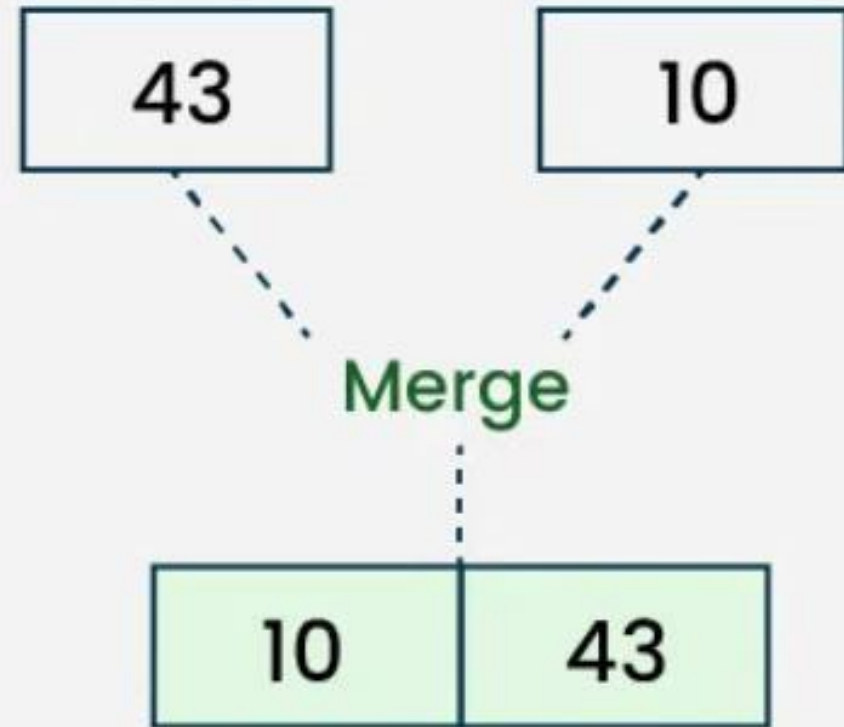
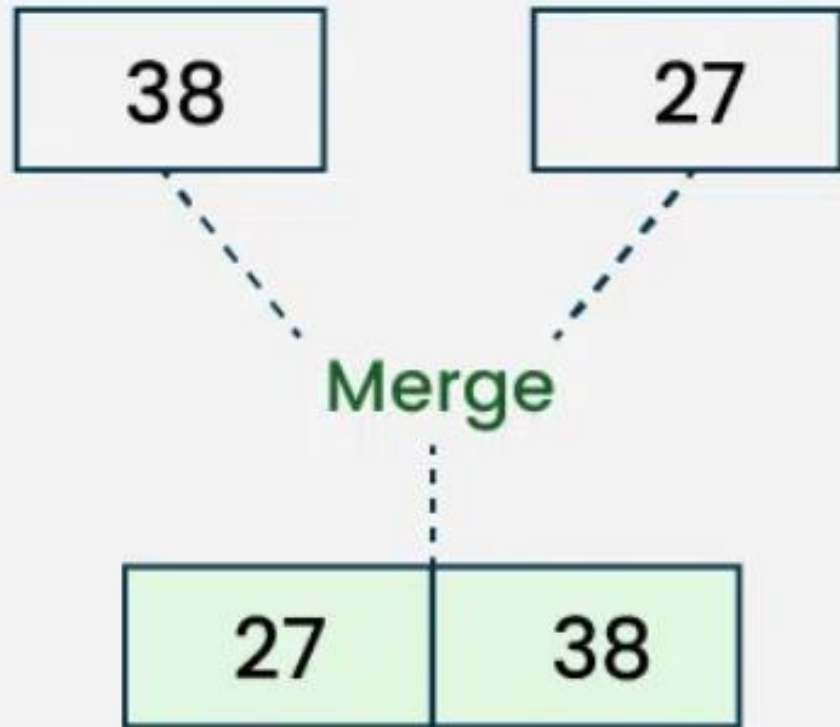
Partition





Step 3

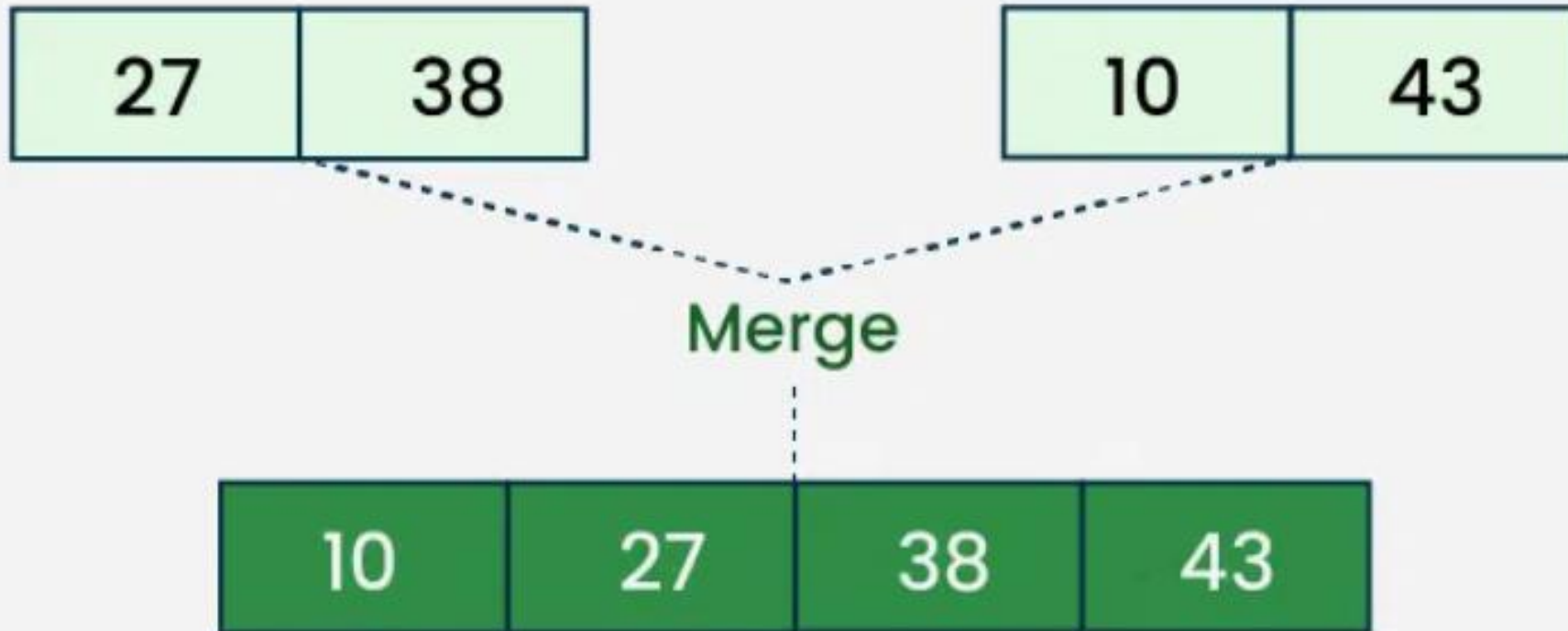
Merging unit length cells into sorted subarrays



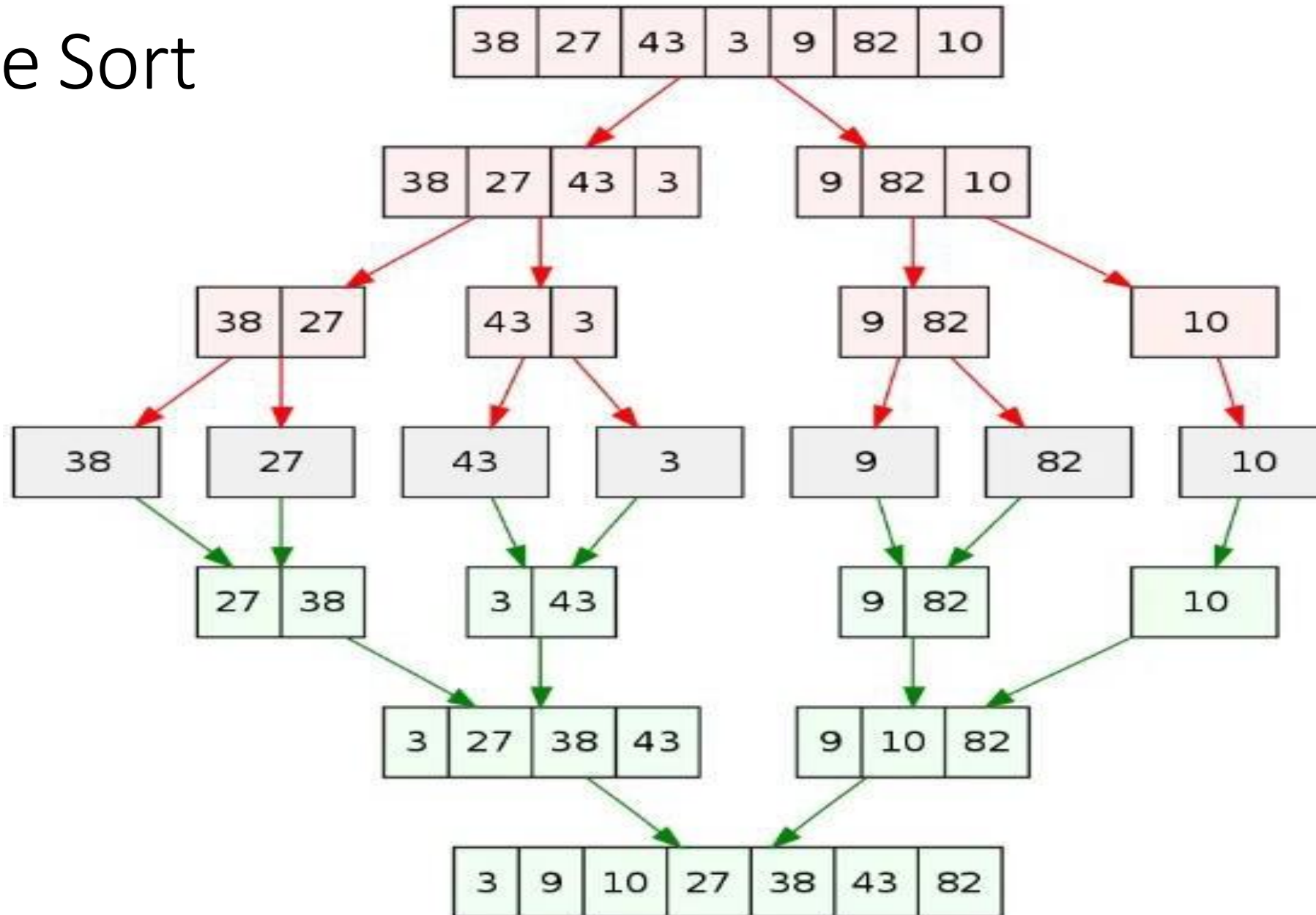


Step 4

Merging sorted subarrays into the sorted array



Merge Sort



merge sort diagram

Merge Sort

- So there are three steps

1. Divide
2. Conquer
3. Merge

- **Illustration of Merge Sort:**

- Let's sort the array or list **[38, 27, 43, 10]** using Merge Sort

Merge Sort

- *Let's look at the working of above example:*
- ***Divide:***
- ***[38, 27, 43, 10] is divided into [38, 27] and [43, 10] .***
- ***[38, 27] is divided into [38] and [27] .***
- ***[43, 10] is divided into [43] and [10] .***

Merge Sort

- **Conquer:**
- **[38]** is already sorted.
- **[27]** is already sorted.
- **[43]** is already sorted.
- **[10]** is already sorted.

Merge Sort

- ***Merge:***
- ***Merge [38] and [27] to get [27, 38] .***
- ***Merge [43] and [10] to get [10,43] .***
- ***Merge [27, 38] and [10,43] to get the final sorted list [10, 27, 38, 43]***
- ***Therefore, the sorted list is [10, 27, 38, 43]***

Merge Sort

- **How it works:**

1. Divide the unsorted array into two sub-arrays, half the size of the original.
2. Continue to divide the sub-arrays as long as the current piece of the array has more than one element.
3. Merge two sub-arrays together by always putting the lowest value first.
4. Keep merging until there are no sub-arrays left.

Merge Sort : Algorithm

- In the following algorithm, **arr** is the given array, **beg** is the starting element, and **end** is the last element of the array.

MERGE_SORT(arr, beg, end)

if beg < end

 set mid = (beg + end)/2

 MERGE_SORT(arr, beg, mid)

 MERGE_SORT(arr, mid + 1, end)

 MERGE (arr, beg, mid, end)

end of **if**

END MERGE_SORT

Merge Sort

- The important part of the merge sort is the **MERGE** function.
- This function performs the merging of two sorted sub-arrays that are **A[beg...mid]** and **A[mid+1...end]**, to build one sorted array **A[beg...end]**.
- So, the inputs of the **MERGE** function are **A[], beg, mid, and end**.


```
public static void main(String[] args) {  
    int[] array = {12, 11, 13, 5, 6, 7};  
    System.out.println("Unsorted Array: " + Arrays.toString(array));  
  
    mergeSort(array, 0, array.length - 1);  
  
    System.out.println("Sorted Array: " + Arrays.toString(array));  
}
```

```
// Function to perform merge sort
private static void mergeSort(int[] array, int left, int right) {
    if (left < right) {
        // Find the middle point
        int mid = left + (right - left) / 2;

        // Recursively sort first and second halves
        mergeSort(array, left, mid);
        mergeSort(array, mid + 1, right);

        // Merge the sorted halves
        merge(array, left, mid, right);
    }
}
```

```
// Function to merge two subarrays
private static void merge(int[] array, int left, int mid, int right) {
    // Find the sizes of the two subarrays
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int[] leftArray = new int[n1];
    int[] rightArray = new int[n2];

    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
        leftArray[i] = array[left + i];
    for (int j = 0; j < n2; j++)
        rightArray[j] = array[mid + 1 + j];
```

```
// Merge the temporary arrays
int i = 0, j = 0, k = left;
while (i < n1 && j < n2) {
    if (leftArray[i] <= rightArray[j]) {
        array[k] = leftArray[i];
        i++;
    } else {
        array[k] = rightArray[j];
        j++;
    }
    k++;
}
```

```
// Copy remaining elements of leftArray, if any
while (i < n1) {
    array[k] = leftArray[i];
    i++;
    k++;
}
// Copy remaining elements of rightArray, if any
while (j < n2) {
    array[k] = rightArray[j];
    j++;
    k++;
}
}
```

Merge Sort

```
// Implementation of merge sort in Java
import java.util.Arrays;

public class MergeSortv1
{
    // Function to merge two subarrays
    private static void merge(int[] array, int left, int mid, int right)
    {
        // Find the sizes of the two subarrays
        int n1 = mid - left + 1;
        int n2 = right - mid;

        // Create temporary arrays
        int[] leftArray = new int[n1];
        int[] rightArray = new int[n2];

        // Copy data to temporary arrays
        for (int i = 0; i < n1; i++)
            leftArray[i] = array[left + i];
        for (int j = 0; j < n2; j++)
            rightArray[j] = array[mid + 1 + j];

        // Merge the temporary arrays
        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            if (leftArray[i] <= rightArray[j]) {
                array[k] = leftArray[i];
                i++;
            } else {
                array[k] = rightArray[j];
                j++;
            }
            k++;
        }

        // Copy remaining elements of leftArray, if any
        while (i < n1) {
            array[k] = leftArray[i];
            i++;
            k++;
        }

        // Copy remaining elements of rightArray, if any
        while (j < n2) {
            array[k] = rightArray[j];
            j++;
            k++;
        }

        // Merge the sorted halves
        merge(array, left, mid, right);
    }

    public static void main(String[] args) {
        int[] array = {12, 11, 13, 5, 6, 7};
        System.out.println("Unsorted Array: " + Arrays.toString(array));

        mergeSort(array, 0, array.length - 1);

        System.out.println("Sorted Array: " + Arrays.toString(array));
    }

    // Function to perform merge sort
    private static void mergeSort(int[] array, int left, int right) {
        if (left < right) {
            // Find the middle point
            int mid = left + (right - left) / 2;

            // Recursively sort first and second halves
            mergeSort(array, left, mid);
            mergeSort(array, mid + 1, right);
        }
    }
}
```

Merge Sort

- **Complexity Analysis of Merge Sort:**
- **Time Complexity:**
 - **Best Case:** $O(n \log n)$, When the array is already sorted or nearly sorted.
 - **Average Case:** $O(n \log n)$, When the array is randomly ordered.
 - **Worst Case:** $O(n \log n)$, When the array is sorted in reverse order.
- **Auxiliary Space:** $O(n)$, Additional space is required for the temporary array used during merging.

Merge Sort

- Advantages of Merge Sort:
- Stability : Merge sort is a stable sorting algorithm, which means it maintains the relative order of equal elements in the input array.
- Guaranteed worst-case performance: Merge sort has a worst-case time complexity of $O(N \log N)$, which means it performs well even on large datasets.
- Simple to implement: The divide-and-conquer approach is straightforward.
- Naturally Parallel : We independently merge subarrays that makes it suitable for parallel processing.

Merge Sort

- Disadvantages of Merge Sort:
- Space complexity: Merge sort requires additional memory to store the merged sub-arrays during the sorting process.
- Not in-place: Merge sort is not an in-place sorting algorithm, which means it requires additional memory to store the sorted data. This can be a disadvantage in applications where memory usage is a concern.
- Slower than QuickSort in general. QuickSort is more cache friendly because it works in-place.

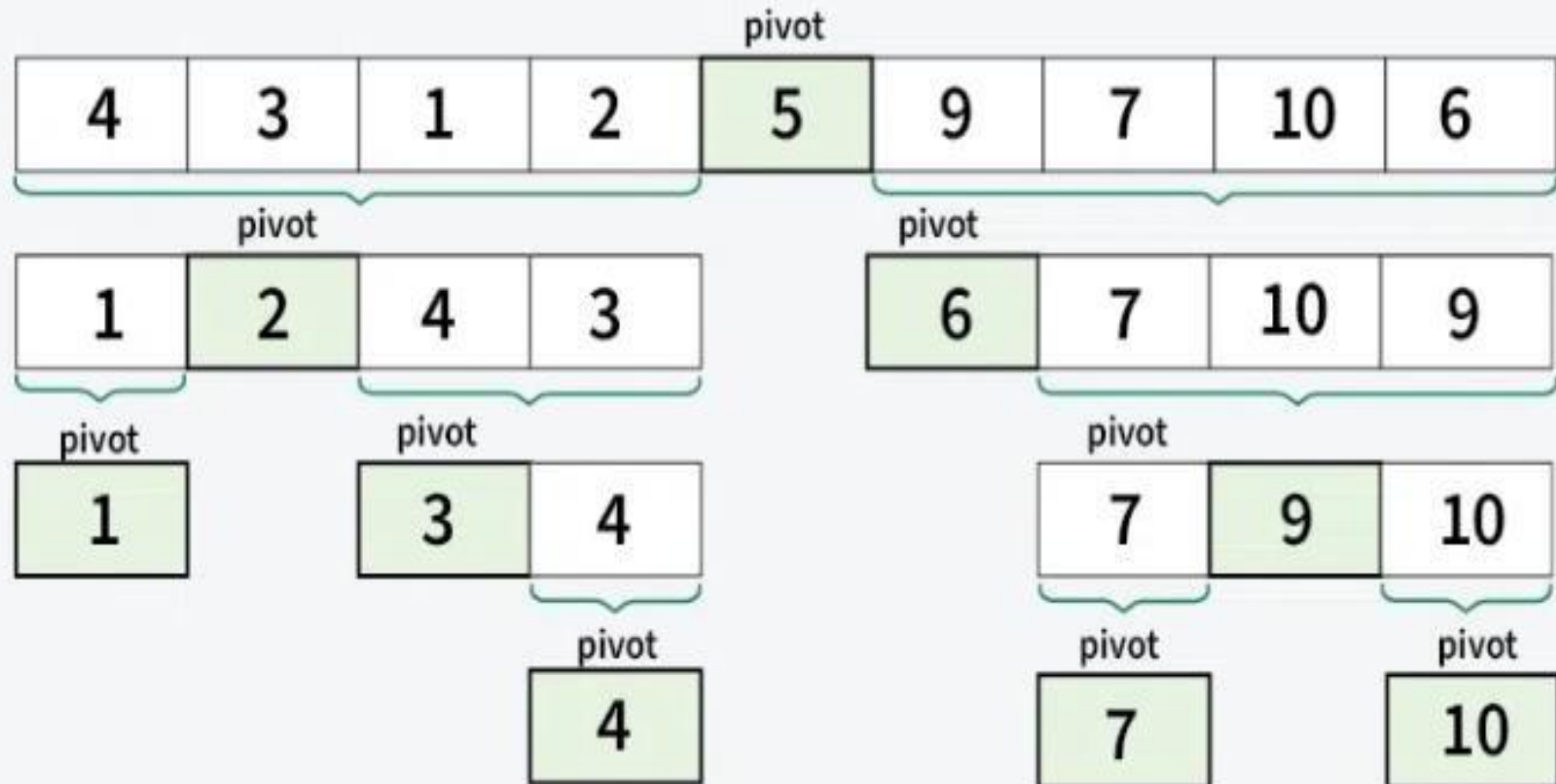
Quick Sort

- **How does QuickSort Algorithm work?**
-
- QuickSort works on the principle of **divide and conquer**, breaking down the problem into smaller sub-problems.

Quick Sort

- There are mainly three steps in the algorithm:
- **Choose a Pivot:** Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).
- **Partition the Array:** Rearrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right. The pivot is then in its correct position, and we obtain the index of the pivot.
- **Recursively Call:** Recursively apply the same process to the two partitioned sub-arrays (left and right of the pivot).
- **Base Case:** The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

Here, we have represented the recursive call after each partitioning step of the array.



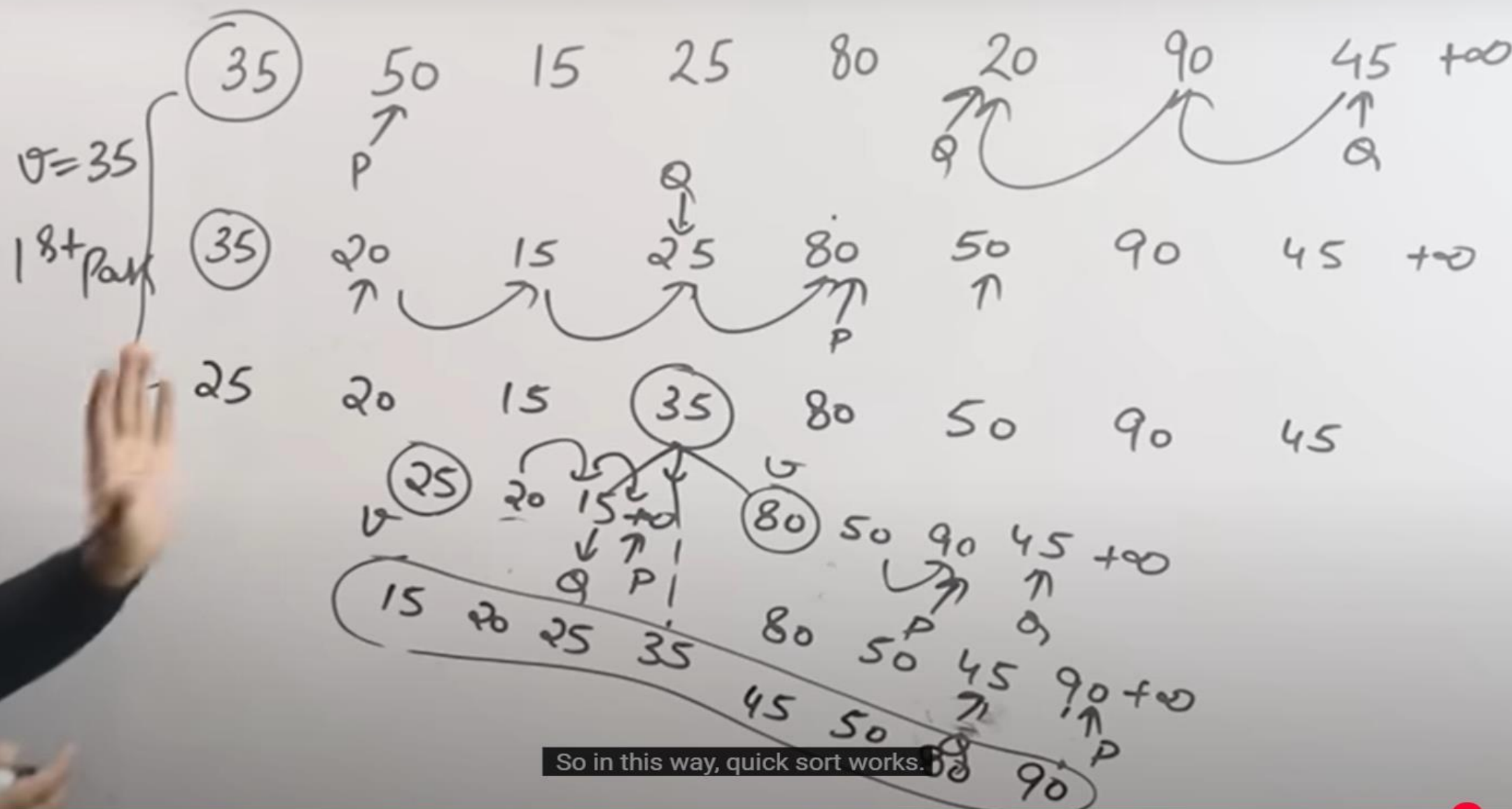
Quick Sort (DAC)

$V=35$

(35)	50	15	25	80	20	90	45
	P ↗						↑ Q

Quick Sort (DAC)

35 5 4 3 2 1



So in this way, quick sort works.

Quick Sort

- **Quick Sort Algorithm with Pointer p, q and Leftmost Pivot**

Quick Sort Algorithm with Pointer p, q and Leftmost Pivot

Purpose:

To sort an array efficiently using the **Quick Sort** algorithm with the pivot as the leftmost element.

Input:

- An array `arr[]` of length `n`.

Output:

- A sorted array in ascending order.

Algorithm:

1. Quick Sort Algorithm (arr[], low, high)

. Steps:

1.If `low < high`:

- . Partition the array using the `partition()` method.
- . Recursively apply Quick Sort to:
 - . Left subarray: `arr[low]` to `arr[q - 1]`
 - . Right subarray: `arr[q + 1]` to `arr[high]`

Else

Exit.

Partition Algorithm Partition (arr,low,high)

- **Purpose:** Partition the array such that elements smaller than pivot go to the left, and elements larger go to the right.
- **Steps:**
 1. Choose the pivot as the **leftmost element** ($arr[p]$).
 2. Initialize:
 - p as the start index.
 - q as the end index.
 3. Traverse until $p < q$:
 - Increment p until $arr[p] > pivot$.
 - Decrement q until $arr[q] < pivot$.
 - If $p < q$, swap $arr[p]$ and $arr[q]$.

Swap $arr[low]$ (pivot) with $arr[q]$ to place the pivot at the correct position

Quick Sort Pseudocode

QUICK_SORT(arr, low, high)

if low < high then

$q \leftarrow \text{PARTITION}(\text{arr}, \text{low}, \text{high})$

 QUICK_SORT(arr, low, $q - 1$)

 QUICK_SORT(arr, $q + 1$, high)

end if

Pseudocode:

Partition Pseudocode

PARTITION(arr, low, high)

$\text{pivot} \leftarrow \text{arr}[\text{low}]$

$p \leftarrow \text{low} + 1$

$q \leftarrow \text{high}$

 while $p \leq q$ do

 while $p \leq \text{high}$ AND $\text{arr}[p] \leq \text{pivot}$ do

$p \leftarrow p + 1$

 end while

 while $q \geq \text{low}$ AND $\text{arr}[q] > \text{pivot}$ do

$q \leftarrow q - 1$

 end while

 if $p < q$ then

 SWAP(arr[p], arr[q])

 end if

end while

SWAP(arr[low], arr[q])

return q

Quick Sort

```

• public class QuicSortV1 {
•
• // Partition method to place pivot at correct position
• public static int partition(int arr[], int low, int high) {
•
•     int pivot = arr[low]; // Pivot as the leftmost element
•
•     int p = low + 1;
•
•     int q = high;
•
•
•
•     while (p <= q) {
•
•         // Move p to the right while arr[p] <= pivot
•
•         while (p <= high && arr[p] <= pivot) {
•
•             p++;
•
•         }
•
•
•
•         // Move q to the left while arr[q] > pivot
•
•         while (q >= low && arr[q] > pivot) {
•
•             q--;
•
•         }
•
•
•
•         // Swap arr[p] and arr[q] if p < q
•
•         if (p < q) {
•
•             swap(arr, p, q);
•
•             }
•
•         }
•
•         // Swap pivot with arr[q] to place pivot in the right position
•
•         swap(arr, low, q);
•
•         return q;
•
•     }
•
• // Quick sort method
• public static void quickSort(int arr[], int low, int high) {
•
•     if (low < high) {
•
•         int q = partition(arr, low, high);
•
•
•
•         // Sort left and right subarrays recursively
•
•         quickSort(arr, low, q - 1);
•
•         quickSort(arr, q + 1, high);
•
•     }
•
• }
•
• // Swap method
•
• public static void swap(int arr[], int i, int j) {
•
•     int temp = arr[i];
•
•     arr[i] = arr[j];
•
•     arr[j] = temp;
•
• }
•
• // Main method
• public static void main(String[] args) {
•
•     int arr[] = {10, 80, 30, 90, 40, 50, 70};
•
•     int n = arr.length;
•
•
•     System.out.println("Original Array:");
•
•     printArray(arr);
•
•
•
•     quickSort(arr, 0, n - 1);
•
•
•     System.out.println("\nSorted Array:");
•
•     printArray(arr);
•
• }
•
•
•
• // Print array method
• public static void printArray(int arr[]) {
•
•     for (int i : arr) {
•
•         System.out.print(i + " ");
•
•     }
•
•     System.out.println();
•
• }
•
•
•
• run:
•
• Original Array:
•
• 10 80 30 90 40 50 70
•
•
•
• Sorted Array:
•
• 10 30 40 50 70 80 90
•
• BUILD SUCCESSFUL (total time: 0 seconds)

```

Quick Sort

- **Complexity Analysis of Quick Sort**
- **Time Complexity:**
- **Best Case:** ($\Omega(n \log n)$), Occurs when the pivot element divides the array into two equal halves.
- **Average Case** ($\theta(n \log n)$), On average, the pivot divides the array into two parts, but not necessarily equal.
- **Worst Case:** ($O(n^2)$), Occurs when the smallest or largest element is always chosen as the pivot (e.g., sorted arrays).
- **Auxiliary Space:** $O(n)$, due to recursive call stack
- Please refer [Time and Space Complexity Analysis of Quick Sort](#) for more details.

Quick Sort

- **Advantages of Quick Sort**

- It is a divide-and-conquer algorithm that makes it easier to solve problems.
- It is efficient on large data sets.
- It has a low overhead, as it only requires a small amount of memory to function.
- It is Cache Friendly as we work on the same array to sort and do not copy data to any auxiliary array.
- Fastest general purpose algorithm for large data when stability is not required.
- It is [tail recursive](#) and hence all the [tail call optimization](#) can be done.

Quick Sort

- **Disadvantages of Quick Sort**
- It has a worst-case time complexity of $O(n^2)$, which occurs when the pivot is chosen poorly.
- It is not a good choice for small data sets.
- It is not a stable sort, meaning that if two elements have the same key, their relative order will not be preserved in the sorted output in case of quick sort, because here we are swapping elements according to the pivot's position (without considering their original positions).

Quick Sort

- **Applications of Quick Sort**

- Efficient for sorting large datasets with $O(n \log n)$ average-case time complexity.
- Used in partitioning problems like finding the k th smallest element or dividing arrays by pivot.
- Integral to randomized algorithms, offering better performance than deterministic approaches.
- Applied in cryptography for generating random permutations and unpredictable encryption keys.
- Partitioning step can be parallelized for improved performance in multi-core or distributed systems.
- Important in theoretical computer science for analyzing average-case complexity and developing new techniques.